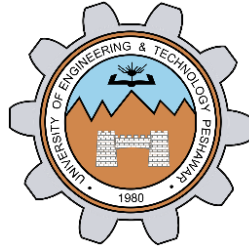


A Digital Lock

LAB # 10



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

23rd May 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives:

This lab will enable students to:

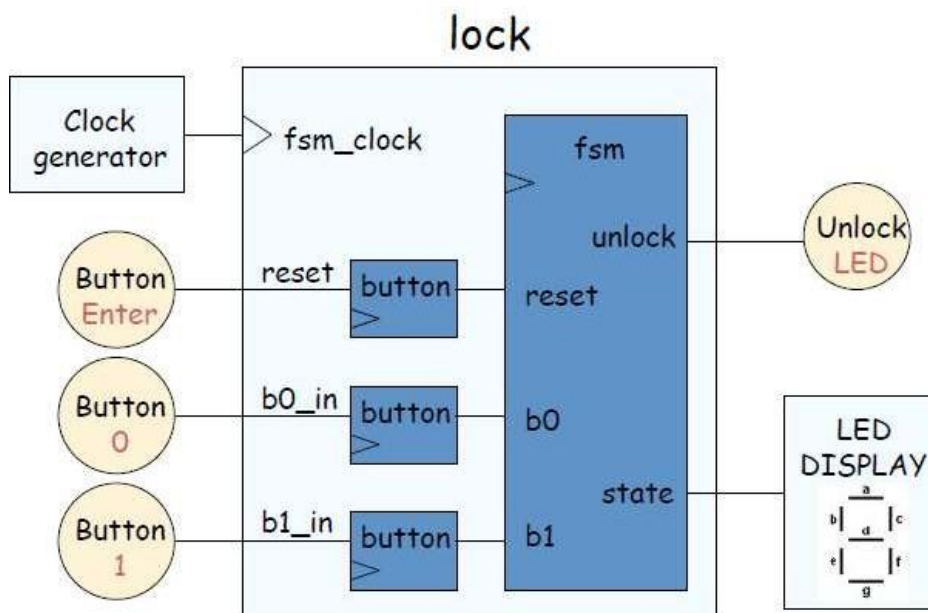
- Build an electronic combination lock with reset button, two number buttons (0 and 1), and an unlock output. The combination should be “01011”

Software used:

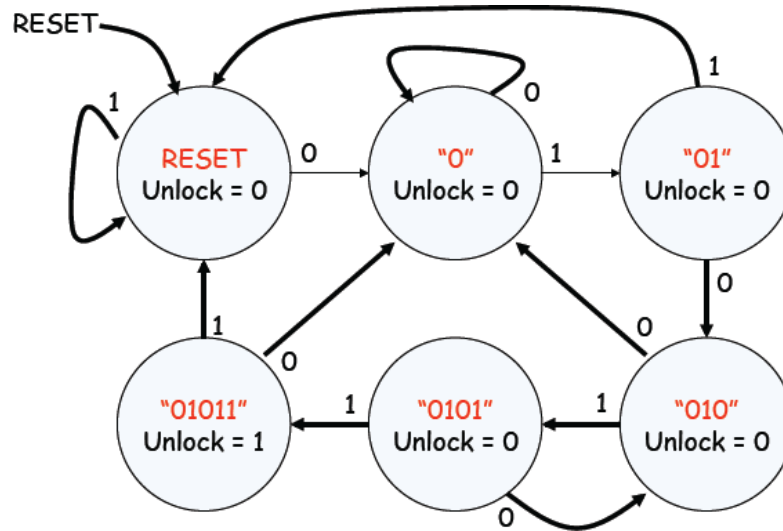
- Xilinx ISE

Block Diagram:

A combinational digital lock has three input buttons for Reset, entering a “0” and entering a “1” and output button UNLOCK and state which shows in which state the machine is currently in. The state output is connected to the seven segment display on the S6BOARD. The Module button in the below diagram is an abstraction of the synchronizer and level to pulse converter from the previous lab. The state transition diagram is given in the following figures.



State Transition Diagram:



Lab Task:

Change the functionality of the lock such that it unlocks on the sequence of 11011.

Code:

```
Design
View: Implementation Simulation
Hierarchy
Task1
xc6slx9-2csg324
Top (Top.v)
divider - clock_divider (clock_divider.v)
my_fsm - lock_fsm2 (lock_fsm2.v)
b0 - btn_module (btn_module.v)
b1 - btn_module (btn_module.v)
b2 - btn_module (btn_module.v)
BCD_to_SevenSeg - BCD_to_SevenSeg (BCD_to_SevenSeg.v)
No Processes Running
Processes: Top
Design Summary/Reports
Design Utilities
User Constraints
Create Timing Constraints
I/O Pin Planning (PlanAhead) - Pre-Synthe...
I/O Pin Planning (PlanAhead) - Post-Synthe...
Floorplan Area/I/O/Logic (PlanAhead)
Synthesize - XST

1 module Top( CLK, RST, RST_BTN, BTN0, BTN1, LED, SEVENSEG);
2   input CLK, RST, BTN0, BTN1, RST_BTN;
3
4   output LED;
5   output [7:0]SEVENSEG;
6
7   wire SLOW_CLOCK;
8   wire pulse0, pulse1, RST_Pulse;
9   wire [3:0]bcd;
10
11  clock_divider divider(CLK, SLOW_CLOCK);
12  lock_fsm2 my_fsm( pulse0, pulse1, SLOW_CLOCK, RST_Pulse, LED, bcd);
13
14  btn_module b0(BTN0, SLOW_CLOCK, RST, pulse0);
15  btn_module b1(BTN1, SLOW_CLOCK, RST, pulse1);
16  btn_module b2(RST_BTN, SLOW_CLOCK, RST, RST_Pulse);
17
18  BCD_to_SevenSeg(bcd, SEVENSEG);
19
20 endmodule
```

Figure 1.1 top module

```

1 module clock_divider(
2     input clk_in,
3     output reg clk_out
4 );
5     reg [27:0] counter = 28'd0;
6     parameter divisor = 28'd1000000;
7
8     always @(posedge clk_in) begin
9         counter <= counter + 28'd1;
10
11         if (counter >= (divisor - 1))
12             counter <= 28'd0;
13
14         clk_out <= (counter < divisor/2) ? 1'b1 : 1'b0;
15     end
16 endmodule

```

Figure 1.2: Clock Divider Code

```

21 module lock_fsm2(
22     input btn0,
23     input btn1,
24     input clk,
25     input RST_BTN,
26     output reg led,
27     output reg [3:0]bcd
28 );
29     reg [2:0]state;
30
31     parameter s0 = 3'b000;
32     parameter s1 = 3'b001;
33     parameter s2 = 3'b010;
34     parameter s3 = 3'b011;
35     parameter s4 = 3'b100;
36     parameter s5 = 3'b101;
37
38     always@(posedge clk) begin
39         if(RST_BTN)
40             state <= s0;
41
42         else
43             case(state)
44                 s0: begin
45                     if(btn0)

```

Figure 1.3.1: Digital Lock FSM Code

```
46         state <= s0;
47     else if(btn1)
48         state <= s1;
49     else
50         state <= s0;
51 end
52
53 s1: begin
54     if(btn0)
55         state <= s1;
56     else if(btn1)
57         state <= s2;
58     else
59         state <= s1;
60 end
61
62 s2: begin
63     if(btn0)
64         state <= s3;
65     else if(btn1)
66         state <= s2;
67     else
68         state <= s2;
69 end
70
```

Design Summary | Top.v | clock_divider.v | lock_fsm2.v

Figure 1.3.2: Digital Lock FSM Code

```
70
71 s3: begin
72     if(btn0)
73         state <= s3;
74     else if(btn1)
75         state <= s4;
76     else
77         state <= s3;
78 end
79
80 s4: begin
81     if(btn0)
82         state <= s4;
83     else if(btn1)
84         state <= s5;
85     else
86         state <= s4;
87 end
88
89 s5: begin
90     if(btn0)
91         state <= s5;
92     else if(btn1)
93         state <= s0;
94     else

```

Design Summary | Top.v | clock_divider.v | lock_fsm2.v

Figure 1.3.3: Digital Lock FSM Code

```
94         else
95             state <= s5;
96         end
97     endcase
98
99 end
100
101
102 always@(*)
103     case(state)
104     s0: begin
105         led <= 0;
106         bcd <= 4'b0000;
107     end
108
109     s1: begin
110         led <= 0;
111         bcd <= 4'b0001;
112     end
113
114
115     s2: begin
116         led <= 0;
117         bcd <= 4'b0010;
118     end
```

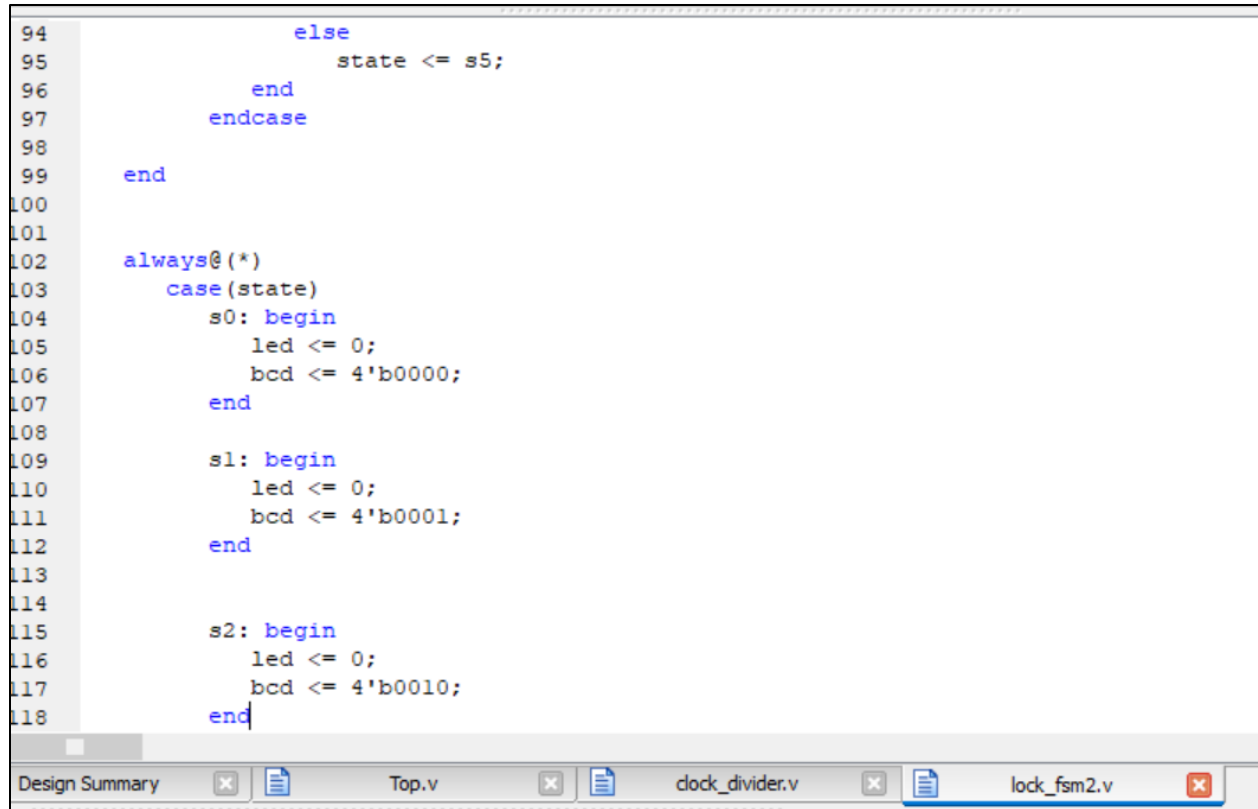


Figure 1.3.4: Digital Lock FSM Code

```
118     end
119
120     s3: begin
121         led <= 0;
122         bcd <= 4'b0011;
123     end
124
125     s4: begin
126         led <= 0;
127         bcd <= 4'b0100;
128     end
129
130     s5: begin
131         led <= 1;
132         bcd <= 4'b0101;
133     end
134 endcase
135
136 endmodule
137
```

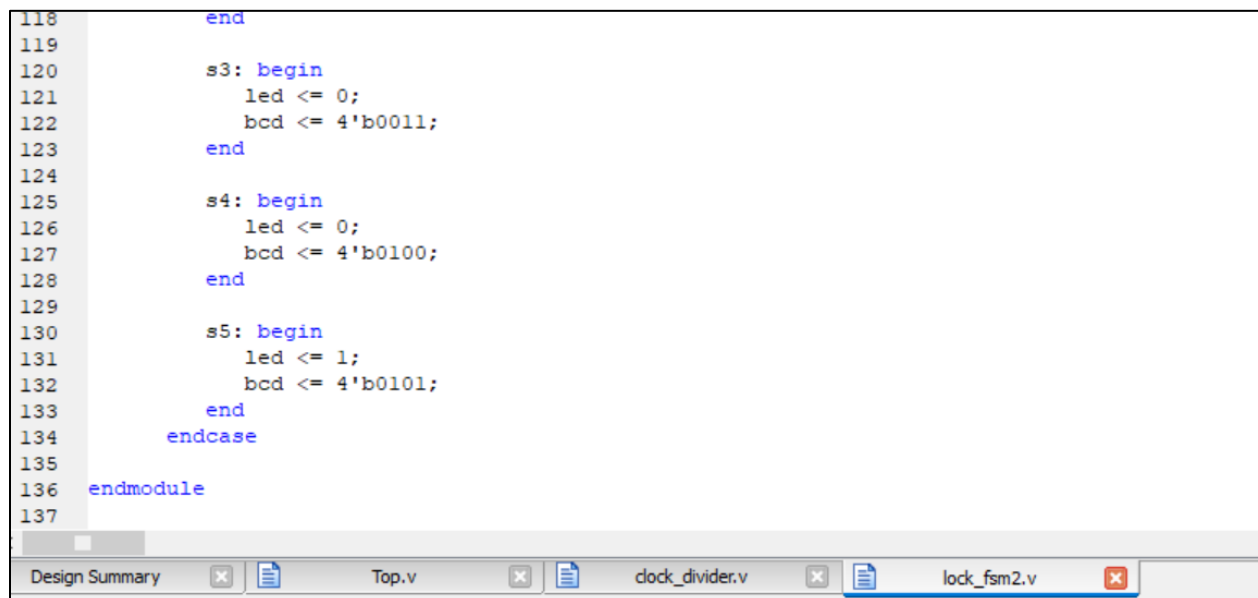


Figure 1.3.5: Digital Lock FSM Code

```
21 module btn_module(  
22     input btn,  
23     input CLK,  
24     input RST,  
25     output pulse  
26 );  
27 wire synch_btn;  
28 synchronizer s1(CLK, btn, RST, synch_btn);  
29 level_to_pulse lp1(synch_btn, CLK, RST, pulse);  
30  
31 endmodule  
32
```

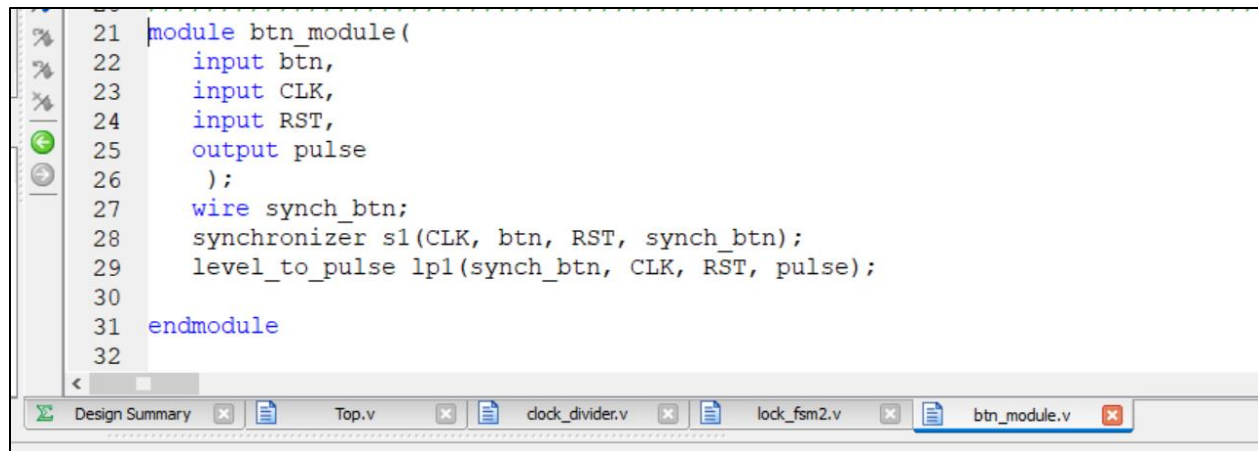


Figure 1.4: btn Module for Push Buttons

```
21 module synchronizer(  
22     input clk,  
23     input btn,  
24     input rst,  
25     output synch_btn  
26 );  
27  
28 wire Q1;  
29  
30 D_FF df1(Q1, btn, clk, rst);  
31 D_FF df2(synch_btn, Q1, clk, rst);  
32  
33 endmodule  
34
```

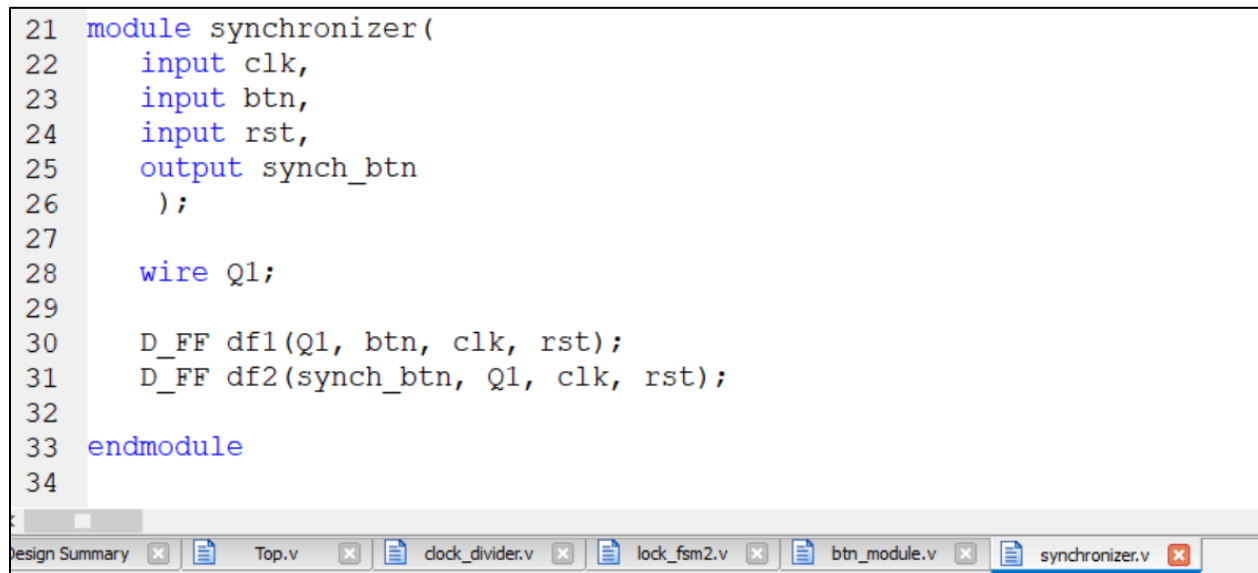


Figure 1.5: Synchronizer Code

```

21 module level_to_pulse(
22     input synch_input,
23     input clk,
24     input rst,
25     output pulse
26 );
27
28     wire Q;
29
30     D_FF df(Q, synch_input, clk, rst);
31     and a(pulse, ~Q, synch_input);
32
33
34 endmodule
35

```

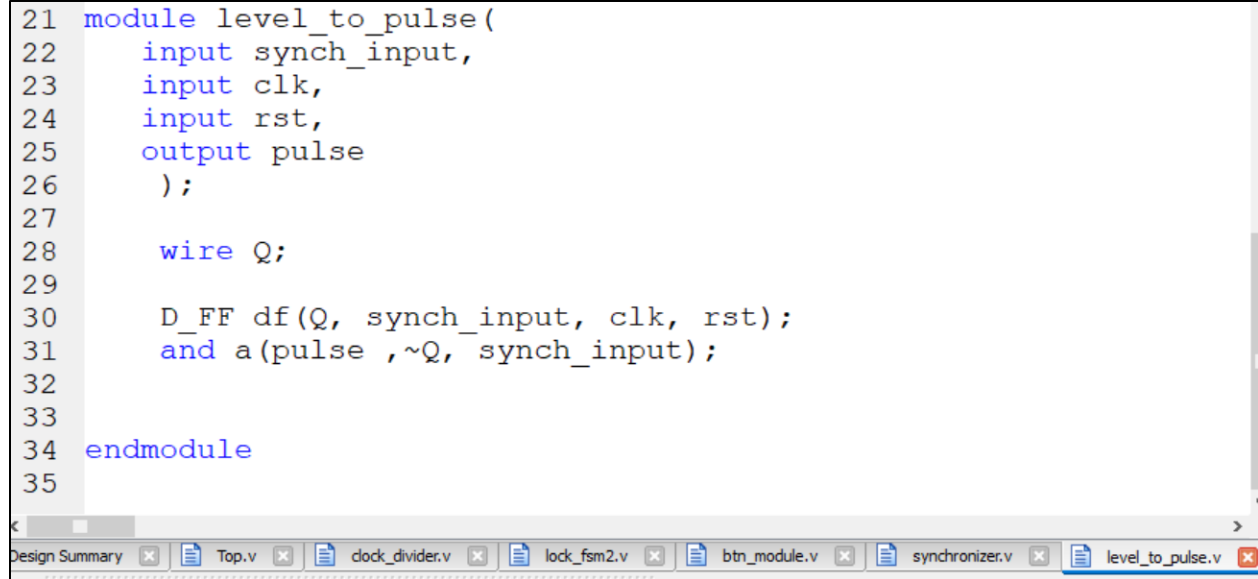


Figure 1.6: Level to Pulse Converter Code

```

1 module D_FF (q, d, clock, reset);
2
3     output q;
4     input d, clock, reset;
5
6     reg q;
7
8     always @(negedge clock or posedge reset)
9     begin
10         if (reset)
11             q = 1'b0;
12         else
13             q = d;
14     end
15
16 endmodule

```

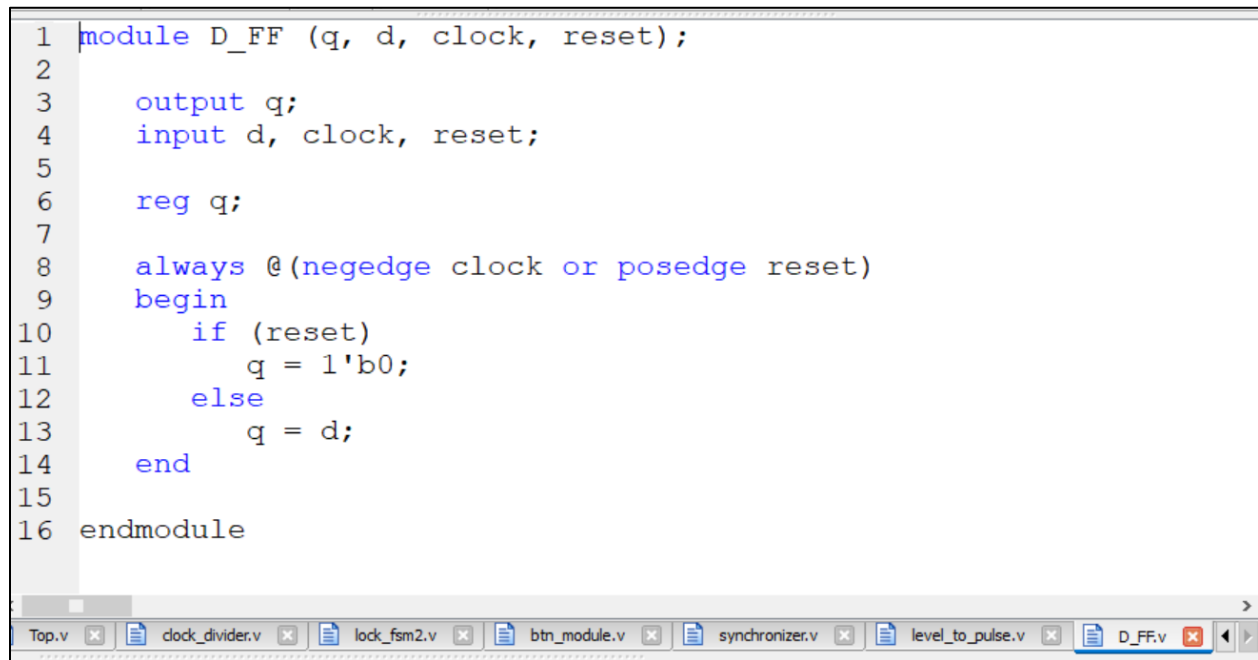


Figure 1.7: D Flip Flop Code

```

1 module BCD_to_SevenSeg(
2     input [3:0] bcd,
3     output reg [7:0] o
4 );
5     always @(*)
6         case(bcd)           //hgfedcba
7             4'b0000: o = 8'b11000000;
8             4'b0001: o = 8'b11111001;
9             4'b0010: o = 8'b10100100;
10            4'b0011: o = 8'b10110000;
11            4'b0100: o = 8'b10011001;
12            4'b0101: o = 8'b10010010;
13            4'b0110: o = 8'b10000010;
14            4'b0111: o = 8'b11111000;
15            4'b1000: o = 8'b10000000;
16            4'b1001: o = 8'b10010000;
17            default: o = 8'b11111111;
18        endcase
19
20 endmodule

```

Figure 1.8: Binary Coded Decimal (BCD) to Seven Segment Decoder

```

1 CONFIG VCCAUX = "3.3" ;
2
3 NET "CLK"          LOC = V10      | IOSTANDARD = LVCMOS33 | PERIOD = 100MHz ;
4 NET "RST"          LOC = C17      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULLUP; #DP
5 NET "BTN0"         LOC = M18      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULLUP; #
6 NET "BTN1"         LOC = L18      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULLUP; #
7 NET "RST_BTN"      LOC = M16      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULLUP
8 NET "LED"          LOC = P15      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #D1
9 NET "SEVENSEG[0]"  LOC = A3       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #a
10 NET "SEVENSEG[1]" LOC = B4       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #b
11 NET "SEVENSEG[2]" LOC = A4       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #c
12 NET "SEVENSEG[3]" LOC = C4       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #d
13 NET "SEVENSEG[4]" LOC = C5       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #e
14 NET "SEVENSEG[5]" LOC = D6       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #f
15 NET "SEVENSEG[6]" LOC = C6       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #g
16 NET "SEVENSEG[7]" LOC = A5       | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #dot
17

```

Figure 1.9: UCF File

Output:

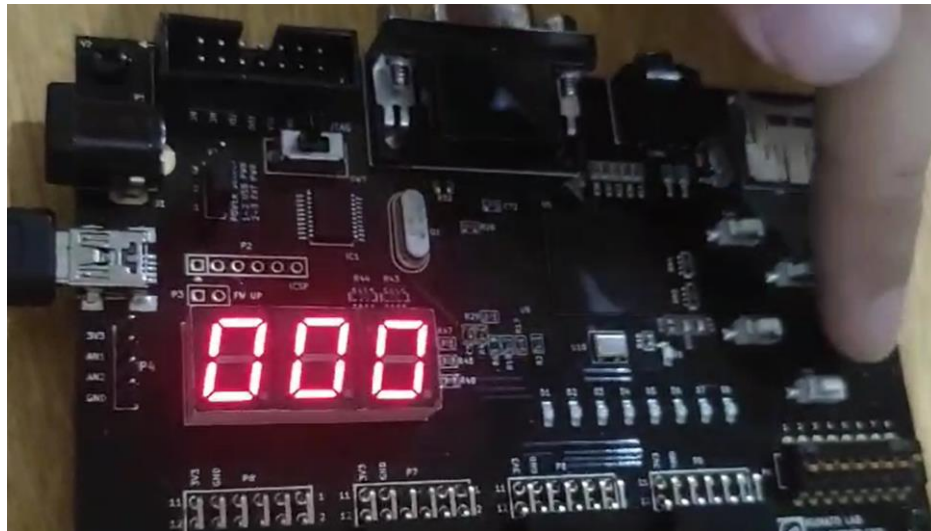


Figure 2.1: Digital Lock Implementation on Spartan6 FPGA Board (State S0)

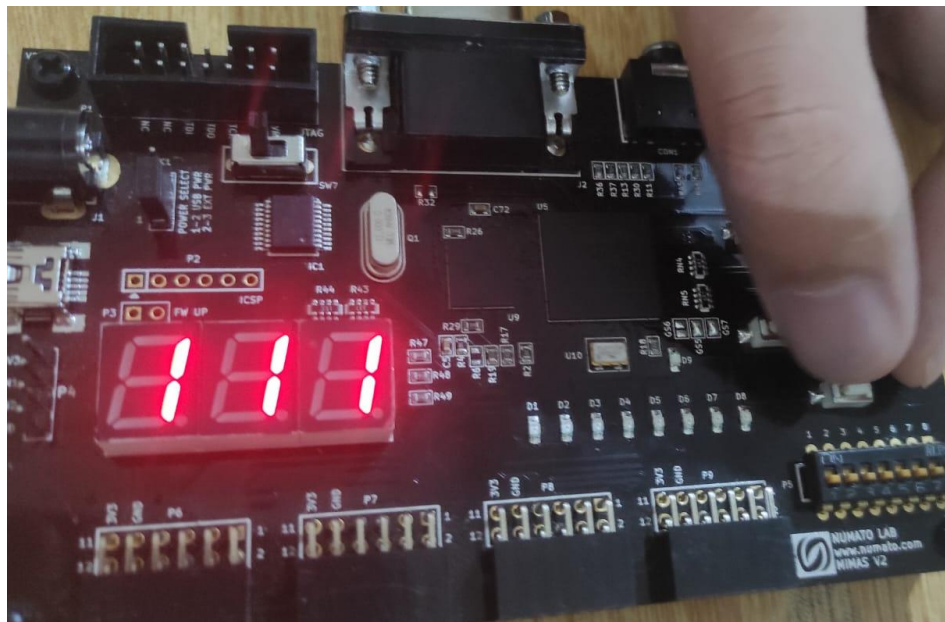


Figure 2.2: Digital Lock Implementation on Spartan6 FPGA Board (State S1)

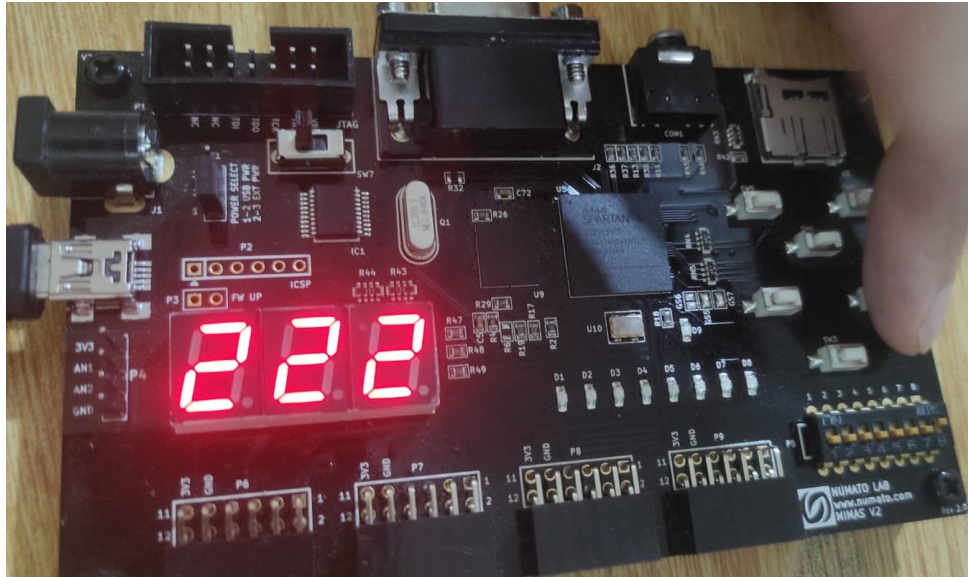


Figure 2.3: Digital Lock Implementation on Spartan6 FPGA Board (State S2)

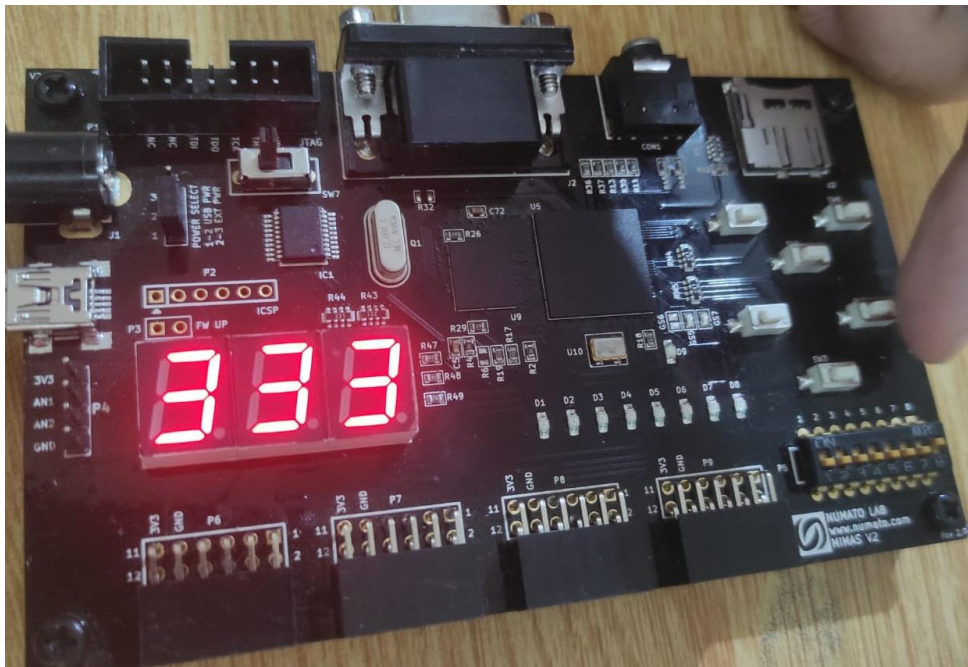


Figure 2.4: Digital Lock Implementation on Spartan6 FPGA Board (State S3)

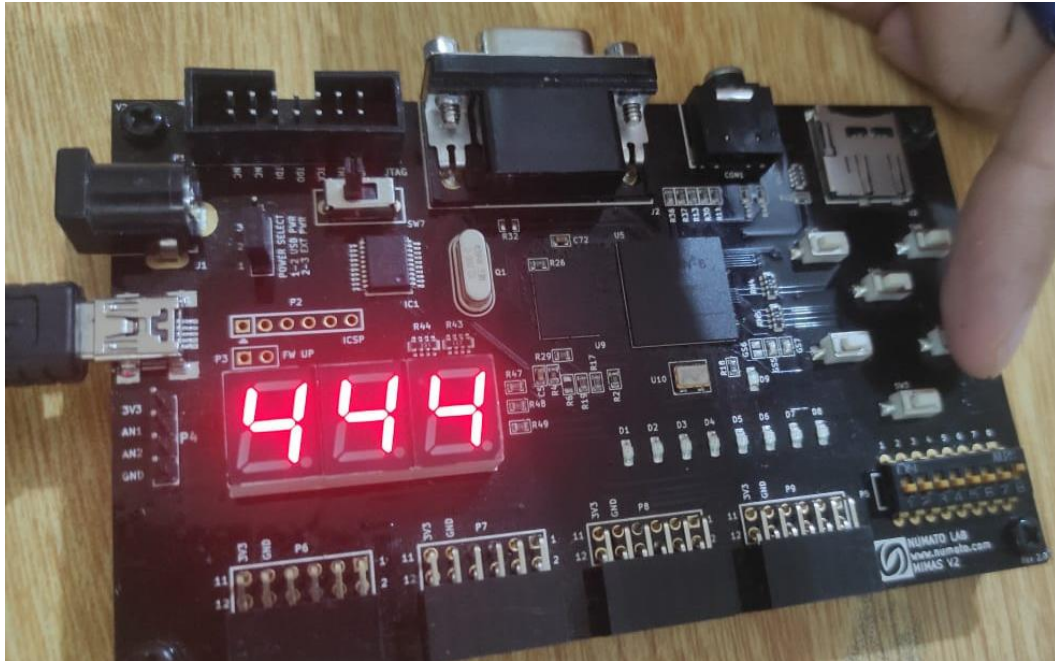


Figure 2.5: Digital Lock Implementation on Spartan6 FPGA Board (State S4)

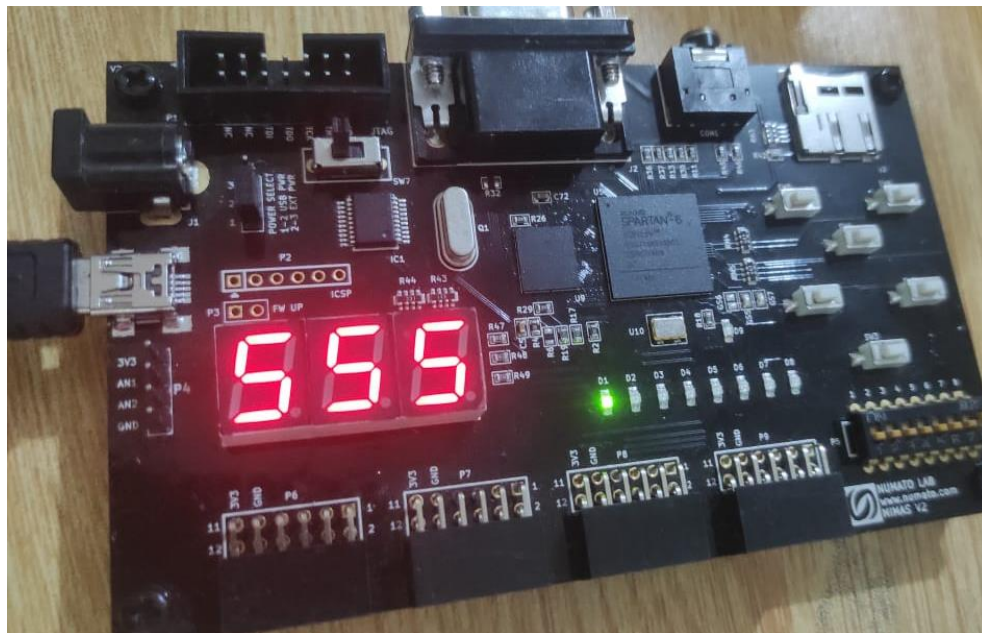


Figure 2.6: Digital Lock Implementation on Spartan6 FPGA Board (State S5)